PYTEST FRAMEWORK

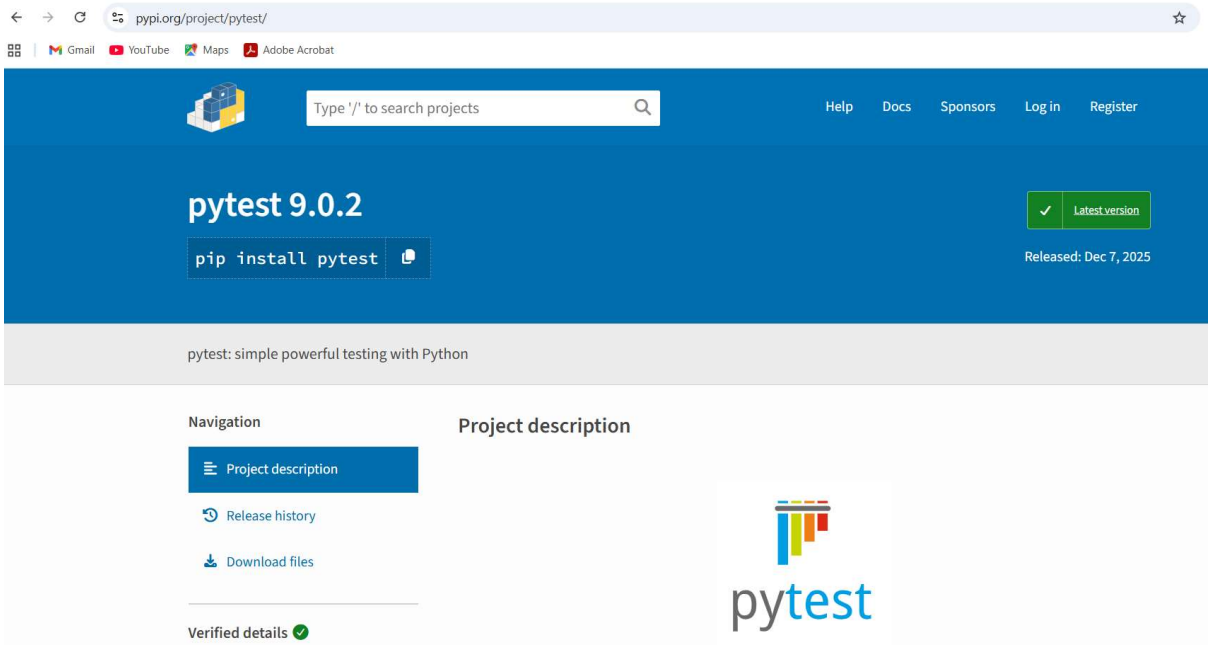
Pytest is an open-source testing framework for Python, highly valued for its simplicity and scalability. It is widely used by developers for writing efficient unit tests, integration tests, and functional tests for APIs, databases, and UI applications.

Key Features

- **Simple Assertions:** Pytest allows the use of standard Python `assert` statements, eliminating the need for special assertion methods provided by other frameworks.
- **Automatic Test Discovery:** The framework automatically identifies test files and functions by searching for those with names starting with `test_` or files named `test_*.py` within a directory, simplifying test execution.
- **Fixtures:** Pytest uses fixtures to manage setup and teardown operations, as well as to create dependencies for tests. This helps in organizing test code and reusing resources efficiently.
- **Parameterization:** It supports running a single test function multiple times with different input arguments (parameterization), which helps achieve broad test coverage with minimal repetitive code.
- **Markers:** Tests can be organized and categorized using markers, allowing users to run specific subsets of tests or skip certain tests during execution.
- **Detailed Reporting:** Pytest provides detailed and informative reports, especially when tests fail, which facilitates debugging.
- **Parallel Execution:** The framework supports running tests in parallel, significantly reducing execution time for large test suites.
- **Rich Plugin Ecosystem:** Pytest has a large and active community that provides a wide range of third-party plugins, extending its functionality.

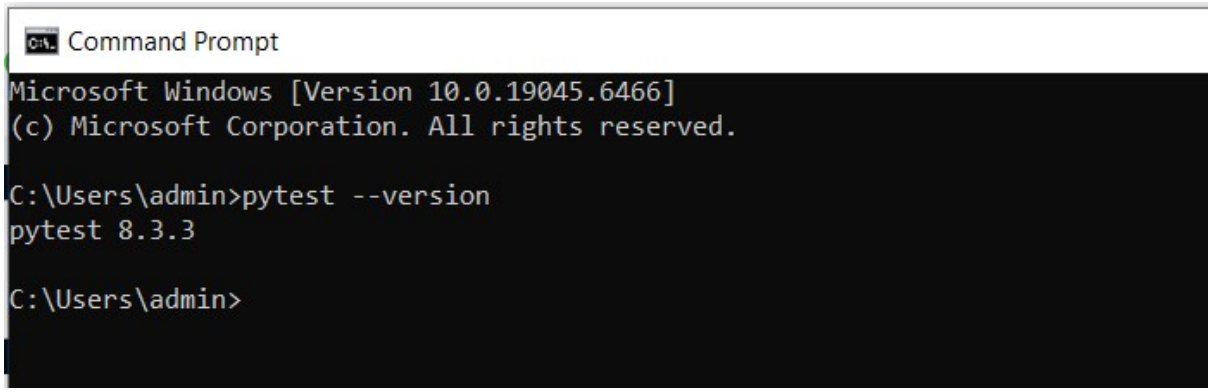
FOR INSTALLATION

pip install pytest



Verify the installation

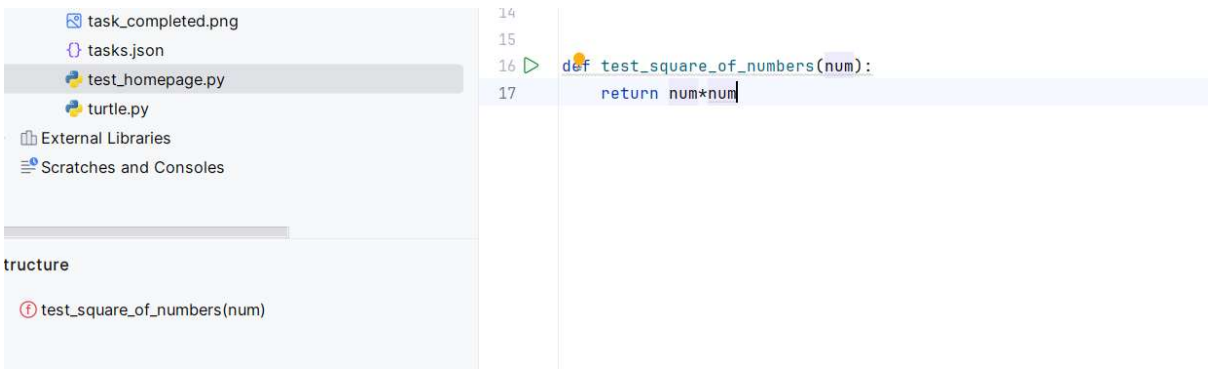
pytest –version



Pytest - Identifying Test files and Functions

- File name should start with test_*.py or *_test.py in the current directory and subdirectories. Pytest automatically identifies those files as test files.
- Pytest requires the test function names to start with **test**. Function names which are not of format **test*** are not considered as test functions by pytest.

Pytest - Starting With Basic Test



test_square.py

```
import math

def test_sqrt():
    num = 25
    assert math.sqrt(num) == 5

def testsquare():
    num = 7
    assert 7*7 == 40

def tesequality():
    assert 10 == 11
```

Pytest - File Execution

To execute the tests from a specific file, use the following syntax –

```
pytest <filename> -v -s
```

Go to project location and open command prompt.

Or

Open command prompt terminal and go to project location – folder, where .py is available, which you want to execute.

Pytest - Substring Matching of Test Names

Suppose we have some method which is having one common word in our .py file.

Ex:-

```
def test_saving_account():
        pass

def test_current_account():
        pass

def test_overdraft_account()”
        pass

def test_square():
        pass
```

def test_prime_numbers():

pass

- **In single test.py file, we have written many methods.**
- **I want to run only those methods which is having "Account" word.**

To execute the tests containing a string in its name we can use the following syntax −

pytest -k <substring> -v –s

Pytest - Grouping the Tests

Pytest allows us to use markers on test functions. Markers are used to set various features/attributes to test functions. Pytest provides many inbuilt markers such as xfail, skip and parametrize. Apart from that, users can create their own marker names. Markers are applied on the tests using the syntax given below −

```
@pytest.mark.<markername>
```

```
1   import pytest
2
3
4   @pytest.mark.test1
5   def test_square_of_numbers(num):
6       return num*num
7
8
9   @pytest.mark.skip
10  def test_double_of_numbers(num):
11      return 2*num
```

Pytest – Fixtures

Fixtures are functions, which will run before each test function to which it is applied.

Fixtures are used to feed some data to the tests such as database connections, URLs to test and some sort of input data.

Therefore, instead of running the same code for every test, we can attach fixture function to the tests and it will run and return the data to the test before executing each test.

- A separate **conftest.py** , we are creating for writing fixtures.
- Name of file should be only **conftest.py**
- A function is marked as a fixture by –

```
@pytest.fixture
```

```
import pytest

@pytest.fixture
def input_value():
    input = 39
    return input

def test_divisible_by_3(input_value):
    assert input_value % 3 == 0

def test_divisible_by_6(input_value):
    assert input_value % 6 == 0
```

Now same code with help of conftest.py file.

conftest.py

```
import pytest

@pytest.fixture
def input_value():
    input = 39
    return input
```

test_division.py

```
import pytest

def test_divisible_by_3(input_value):
    assert input_value % 3 == 0

def test_divisible_by_6(input_value):
    assert input_value % 6 == 0
```

Pytest - Parameterizing Tests

Parameterizing of a test is done to run the test against multiple sets of inputs. We can do this by using the following marker –

```
@pytest.mark.parametrize
```

test_even_or_odd.py

```
import pytest

@pytest.mark.parametrize("number", [1, 2, 3, 4, 5])
def test_is_even(number):
    assert number % 2 == 0, f"{number} is not even"
```

Pytest - Xfail/Skip Tests

- A test is not relevant for some time due to some reasons.
- A new feature is being implemented and we already added a test for that feature.

Pytest will execute the xfailed test, but it will not be considered as part failed or passed tests.

Details of these tests will not be printed even if the test fails (remember pytest usually prints the failed test details).

We can xfail tests using the following marker –

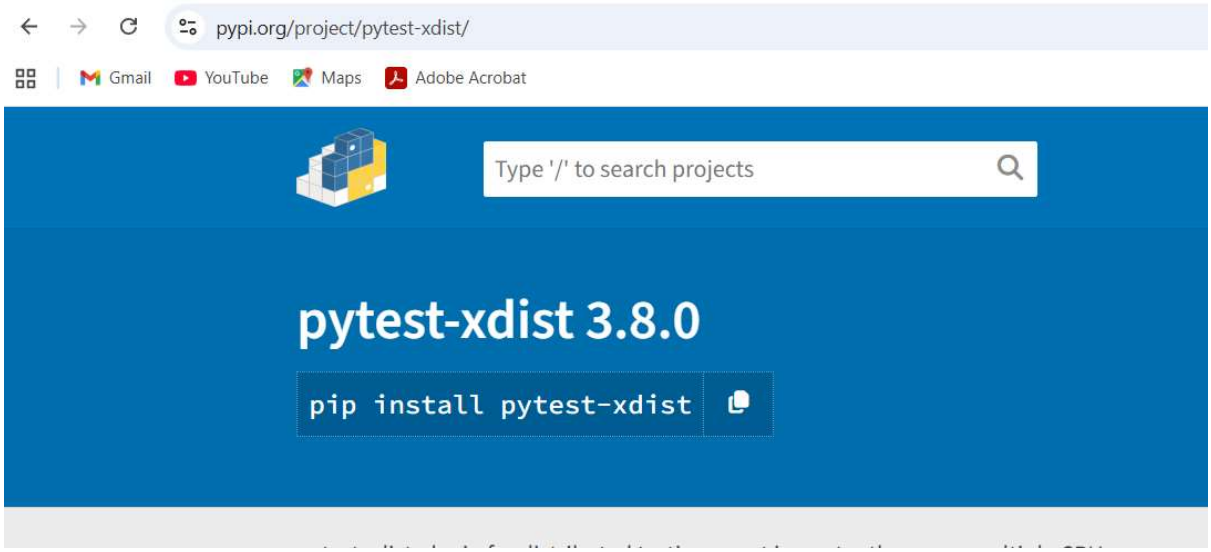
```
@pytest.mark.skip
def test_double_of_numbers(num):
    return 2*num
```

```
@pytest.mark.xfail
def test_double_of_numbers(num):
    return 2*num
```

Run Pytest Tests in Parallel

By default, pytest runs tests in sequential order. In a real scenario, a test suite will have a number of test files and each file will have a bunch of tests. This will lead to a large execution time. To overcome this, pytest provides us with an option to run tests in parallel.

For this, we need to first install the **pytest-xdist plugin.**



Now, we can run tests by using the syntax **pytest -n <num>**

```
pytest -n 3
```

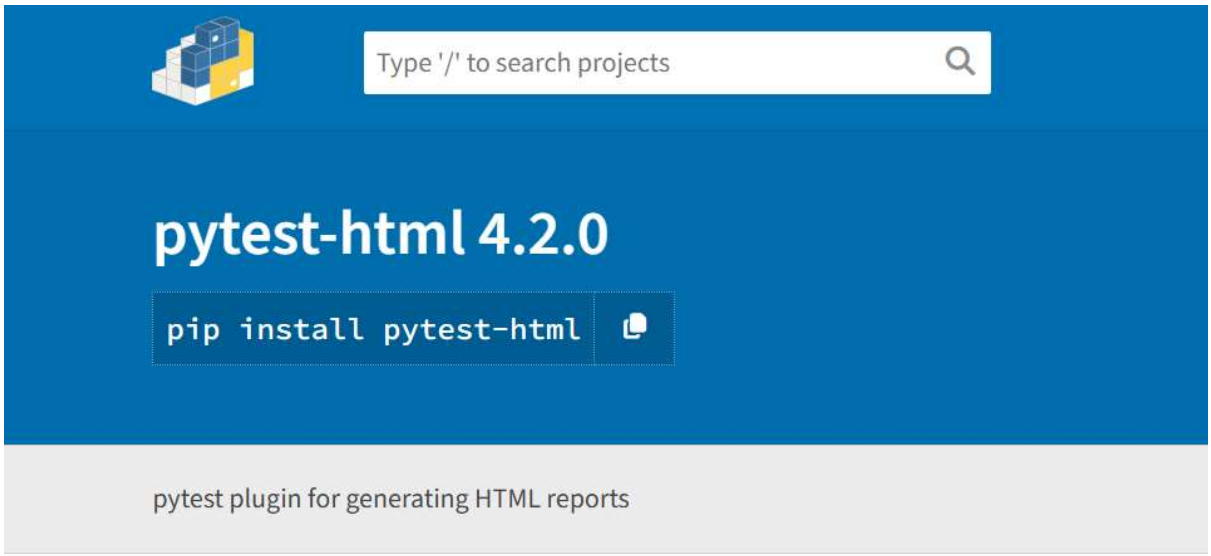
-n <num> runs the tests by using multiple workers, here it is 3.

Summary

- Installing pytest..
- Identifying test files and test functions.
- Executing all test files using pytest v.
- Executing specific file usimng pytest <filename> -v.
- Execute tests by substring matching pytest -k <substring> -v.
- Execute tests based on markers pytest -m <marker_name> -v.
- Creating fixtures using @pytest.fixture.
- conftest.py allows accessing fixtures from multiple files.
- Parametrizing tests using @pytest.mark.parametrize.
- Xfailing tests using @pytest.mark.xfail.
- Skipping tests using @pytest.mark.skip.
- Stop test execution on n failures using pytest --maxfail = <num>.
- Running tests in parallel using pytest -n <num>.

Reporting

To generate an HTML report for your test results with `pytest`, you need to use the pytest-html plugin.




```
# content of test_example.py
def test_pass():
    """This test passes."""
    assert True

def test_fail():
    """This test fails."""
    assert False
```

Run the tests and generate the report: Open your terminal, navigate to the directory containing your test file, and run the following command:

```
pytest --html=report.html
```